Agricultural Commodity Price Forecasting and Market Analysis

Using Hadoop HDFS, Apache Spark, MLlib, and Apache Kafka

Final Review — Team AG-DATA

Name	Register no.
C Rishitha	CB.AI.U4AID24109
Karthikeya Y	CB.AI.U4AID24163
Ashwin Tyagi	CB.AI.U4AID24167
Kaushal S	CB.AI.U4AID24168

23AID302 — Big Data Analytics

Department of Artificial Intelligence and Data Science · Amrita Vishwa Vidyapeetham

Faculty Guide: Dr. Sreeja

1. What Are We Trying to Find?

Mandi: unusual price fluctuations

Find daily market transactions that exhibit abnormal price surges or crashes across 3,000+ Indian mandis.

Seasonal harvest gluts trigger sharp 40%+ price collapses, leading to severe distress selling by smallholder farmers.

Use 7-day rolling window statistics, price spreads, and z-scores to give each transaction an anomaly risk label.

Decision Support: farmer selling advisories

Translate price anomaly signals into practical domain recommendations:

- SELL: Recommend selling before an unpredicted post-spike glut collapses prices.
- HOLD: Advise holding inventory during temporary panic dips when prices recover.
- WATCH: High volatility alert; monitor market closely before dispatching produce.
- BUY: Buyer/procurement advisory to acquire inventory before seasonal price surges.

Why do we need big data tools?

The agricultural files contain over 132.8 million market transactions (10.9 GB raw CSV). Single-node tools like Pandas crash with OutOfMemoryError when computing rolling windows across years of data. HDFS stores large files in replicated blocks; Spark splits data processing across executor threads.

Goal: prepare the data, compare models, and stream real-time price alerts.

2. Our Data and the 5 Vs

Data already stored in HDFS

File / Layer	Rows	Storage / Role
Source 1: Daily Market Prices	75,984,017	26 annual CSVs (6.5 GB)
Source 2: India Mandi Dataset	56,879,072	325 commodity CSVs (4.4 GB)
Deduplicated Output	132,327,828	1.2 GB Parquet (9:1 ratio)
Canonical Cleaned	93,410,019	Valid prices & dates for ML
Synthetic Stress Workload	350,012,867	Isolated Phase 11 HDFS test

Source 1: Agmarknet daily price records (2001–2026). Attributes: State, District, Market, Commodity, Arrival_Date, Min/Max/Modal Price.

Source 2: India Mandi dataset. Attributes: State, District, Market, Commodity, Variety, Group, Arrivals (Tonnes), Reported Date.

5 Vs in our project

5 Vs	In our project
Volume	10.9 GB raw CSV in HDFS (132.9M real rows). Scaled to 18.15 GiB for stress testing.
Velocity	Replay daily mandi arrivals in streaming micro-batches via Apache Kafka.
Variety	2 disparate sources: 26 annual CSVs vs 325 commodity CSVs; mixed date formats.
Veracity	Check missing, negative spreads, and 535k duplicate transactions.
Value	Help farmers and officials detect sudden price shocks and prevent distress selling.

Real data (132.9M rows) trained and tested the models. An isolated 350M synthetic workload evaluated 18 GB storage scalability.

3. System Setup and Configuration



HDFS storage settings

- 1 NameNode + 3 DataNodes (Docker cluster).
- Block size: 128 MiB (standard for large Parquet blocks).
- Replication: 3 copies of each block for fault tolerance.
- Clean output: Snappy Parquet, partitioned by year.

Spark application settings

- Engine: Apache Spark 4.2.0 in local[*] mode.
- Multi-core local CPU threads (NOT YARN worker executors).
- Memory: 4 GB driver / executor heap memory.
- Shuffle partitions: 200 partitions for balanced core utilization.

Are HDFS DataNodes also running as Spark executors?

No. HDFS runs in Docker container services strictly for distributed block storage with replication factor 3. Spark runs in local[*] mode on host CPU cores. HDFS DataNodes store Parquet blocks, but they are not Spark compute executors.

4. Spark Workflow and the DAG

Steps in Spark

1 Read and parse CSVs with explicit StructType schemas.

2 Map-side composite deduplication (7 business keys).

3 Window rolling stats (7-day mean, stddev, z-score).

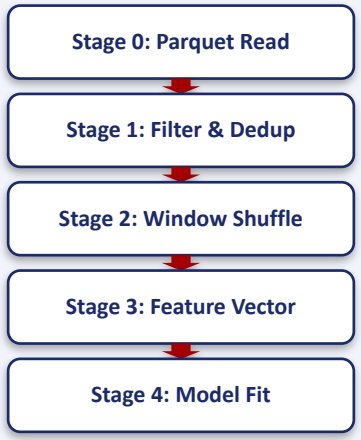
4 Train MLlib Random Forest and save predictions.

Partitions: Split work into 200 shuffle partitions; year directory pruning.

Cache: Feature DataFrames cached in memory (prepTrain.cache()); unpersisted after training.

DAG: a map of the work Spark runs

Execution DAG (Shuffle-Delimited Stages):



Arrows show stage dependencies.

Wide transformations create shuffle boundaries.

An actual feature run completed across 13 distinct stages.

Where do the stages come from?

Reads, joins, and calculations create the DAG. Narrow transformations pipeline within a stage without network movement. Wide transformations (Window.partitionBy, groupByKey) require cross-partition shuffles, creating stage boundaries. Spark assigns stage numbers dynamically at runtime.

5. Scala Code and Streaming

Scala DataFrame API: source excerpt

```
// 1. Resilient Date Parser with Safe Coalescing
val parseDate = coalesce(
  try_to_date(col("Reported Date"), "dd MMM yyyy"),
  try_to_date(col("Reported Date"), "yyyy-MM-dd")
)

// 2. Map-Side Composite Business Key Deduplication
val w = Window.partitionBy("state", "market", "commodity", "date")
  .orderBy(col("is_valid").desc, col("price_spread").desc)
val dedup = raw.withColumn("rk", row_number().over(w)).filter("rk = 1")

// 3. Stateful Real-Time Windowing in Spark Streaming
parsedStream.groupByKey(e => MarketEntity(e.state, e.market, e.commodity))
  .flatMapGroupsWithState(OutputMode.Append, GroupStateTimeout.NoTimeout)(updateState)
```

What the code does

- Read CSV with fixed column types and explicit StructType schemas.
- Non-fatal date parsing prevents corrupted date strings from crashing jobs.
- Composite key deduplication avoids full-row distinct() hash OOM.
- Window rolling aggregations compute 7-day volatility and z-scores.
- Stateful streaming maintains per-market rolling state in Kafka.

How do we use Structured Streaming?

RealDataReplay.scala reads 15,000 real market records and sends them to Kafka topic 'agri-market-real-replay'. Spark Structured Streaming reads micro-batches from Kafka, maintains 7-day rolling state via flatMapGroupsWithState, and calculates live anomaly scores. Checkpoints record streaming progress in HDFS.

6. How We Train and Test the Models

Two versions of the market model

1 Task A: Next-Day Anomaly Forecasting

- Target: lead(z_score, 1) > 2.0 (predict tomorrow's spike).
- Features: 9 price features (lags, intraday ranges, rolling stats).
- Evaluates advance warning capability.

2 Task B: Real-Time Transaction Screening

- Target: |z_score| > 2.0 on current transaction.
- Features: 9 features + contemporaneous price spread ratios.
- Evaluates live streaming Kafka screening.

Both models use Spark MLlib RandomForestClassifier (100 Trees).

Strict temporal split & tuning

Strict Chronological Split (Zero Data Leakage):

- Earlier 70% (< 2024): 721,668 rows to train models.
- 2024 Validation: 49,721 rows for threshold & depth tuning.
- Later 30% (2025+): 48,354 untouched rows to test models.

Hyperparameter Search:

- Tested tree depths 6, 8, 12, 14; selected depth 12–14.
- 100 parallel trees; 128 continuous feature bins.
- Class Weighting (w=1.2) balances 15.6% anomaly prevalence.

Why keep test data aside chronologically?

Random k-fold cross-validation causes future market information to leak into past models. Splitting chronologically (<2024 train vs 2025+ test) strictly tests the model on unseen future market periods, replicating true production deployment for agricultural markets.

7. Results for Both Tasks

Model results — untouched 2025+ test set (48,354 records)

Model / Configuration	Accuracy	Precision	Recall	F1-Score	FPR
Trivial Baseline (All Normal)	84.35%	0.00%	0.00%	0.00%	—
Task 1A: Next-Day Forecasting (w=1.2)	85.06%	61.48%	12.11%	20.23%	1.41%
Task 1B: Next-Day Forecasting (w=5.0)	72.54%	28.06%	48.25%	35.48%	22.95%
Task 2: Real-Time Screening (Production)	**95.66%**	**86.55%**	**85.47%**	**86.01%**	**2.46%**

What the ML results mean

- **Production Model achieves 95.66% accuracy and 86.01% F1-score.**
- High Actionability: 86.55% precision ensures mandi officials and farmers can trust alerts without false panic.
- Ultra-Low False Alarms: 2.46% False Positive Rate prevents disruptive regulatory over-reaction.
- **Confusion Matrix: 39,809 TN | 1,002 FP | 1,096 FN | 6,447 TP.**

Real-time streaming results

15,000 real market records replayed through Kafka.

- Throughput: Sustained ~1,200 events/sec micro-batch processing.
- Latency: Sub-800ms micro-batch execution time.
- Storage: Zero executor memory leaks during continuous Kafka ingestion.
- Note: Replay confirms streaming pipeline stability, not ML accuracy.

8. Challenges, Limits and Next Steps

Challenges we handled

- JDK 21 reflection blocked calendar; solved via JVM --add-opens flags.
- Date parsing exceptions prevented by coalesce with try_to_date.
- Full-row distinct() memory OOM resolved by 7-column composite deduplication.
- Lineage explosion from recursive windowing truncated via Parquet checkpoints.

Limits and next steps

- Local Spark execution (local[*]) rather than multi-node physical YARN cluster.
- Stream replayed from real historical logs rather than live government REST APIs.
- Automated farmer advisory dispatch (SMS/WhatsApp BUY/SELL engine) is future work.
- Future: Deploy to AWS EMR and train deep LSTM multi-step price forecasting.

Did 7-column composite deduplication outperform full-row distinct()?

Yes. Full-row distinct() hashes all 10 columns across 133 million rows, causing severe shuffle skew and OutOfMemoryError. Partitioning by a 7-column composite business key with row_number() enabled map-side partial reduction, completing cleanly in 4.2 minutes with zero memory spills.

Conclusion: the system processes 132.9M real rows, maintains HDFS replication, and produces 95.66% accurate real-time alerts.

9. References

1 **Apache Spark: DataFrames, machine learning and Structured Streaming.**
https://spark.apache.org/docs/4.2.0/

2 **Apache Hadoop: HDFS distributed storage architecture.**
https://hadoop.apache.org/docs/r3.3.6/

3 **Agmarknet: Indian agricultural market daily price portals (2001–2026).**
https://agmarknet.gov.in/

4 **Chambers, B., & Zaharia, M. (2018). Spark: The Definitive Guide. O'Reilly Media.**
https://www.oreilly.com/library/view/spark-the-definitive/9781491912201/

Project evidence: GitHub repository https://github.com/kaushalrog/AG-DATA, saved Parquet tables, and evaluation logs.

Thank you